

1교시: port binding과 localhost 접근

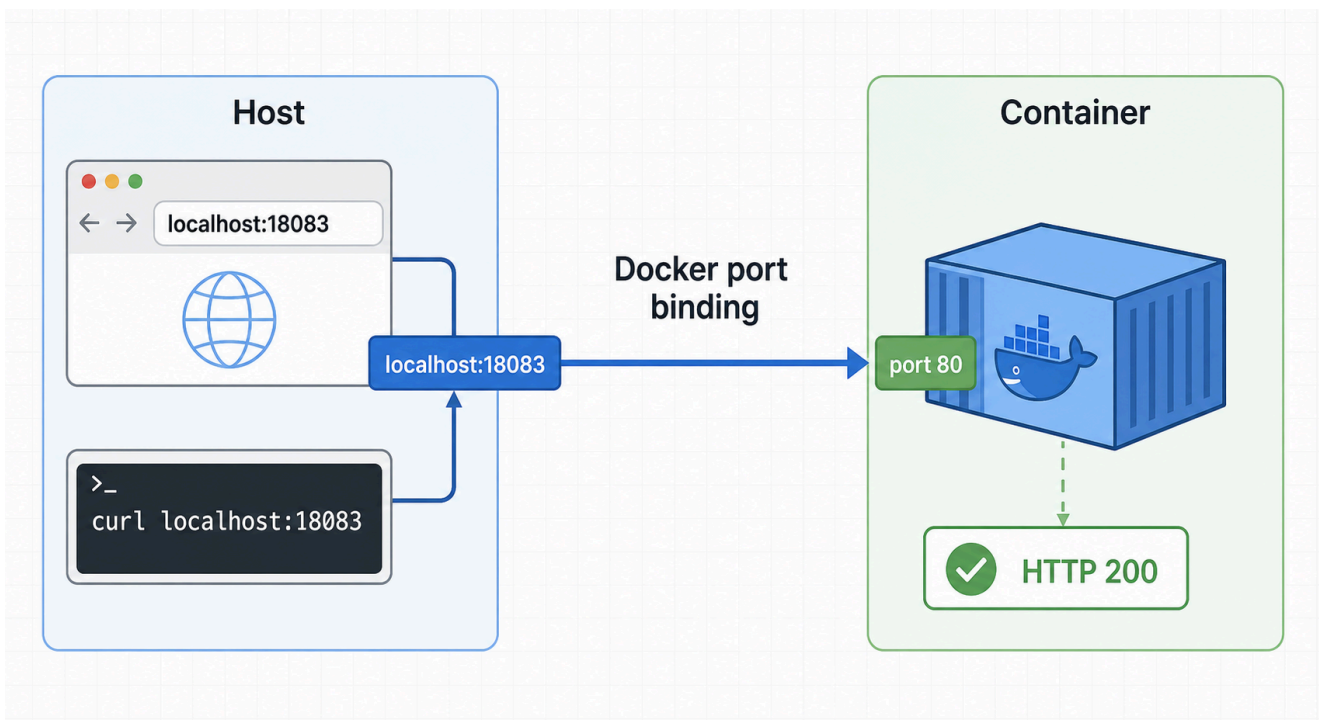
수업 목표

- host port와 container port를 구분한다.
- -p 18083:80이 어떤 방향의 연결인지 설명한다.
- docker ps, curl -I, docker logs로 HTTP service evidence를 확보한다.

50분 흐름

시간	활동	비중	산출
0-5분	Day 2 image/container 복습	설명 10%	image/runtime 구분
5-15분	host port와 container port 설명	설명 20%	port mapping note
15-30분	nginx bind mount container 실행	실행 30%	docker ps evidence
30-40분	HTTP header/body 확인	실행 20%	curl evidence
40-47분	wrong port drill	실행 15%	failure note
47-50분	정리와 다음 교시 연결	설명 5%	cleanup note

Visual 1: Port binding과 localhost



이 이미지는 host의 localhost:18083 요청이 Docker port binding을 지나 container 내부 80번 port로 전달되는 흐름을 보여준다.

핵심 설명

container 내부에서 nginx가 80번 port를 듣고 있어도 host browser가 자동으로 접근할 수 있는 것은 아닙니다. container는 host와 다른 network namespace 안에서 실행된다. -p 18083:80은 host의 18083번 port를 container의 80번 port에 publish한다.

왼쪽 값은 host port이고 오른쪽 값은 container port다. -p 18083:80에서 browser와 curl은 localhost:18083으로 접근한다. nginx는 container 내부에서 80번 port를 사용한다.

EXPOSE 80은 image metadata이고, -p 18083:80은 runtime publish다. Day 2에서 Dockerfile에 EXPOSE 80을 봤더라도 host에서 접속하려면 Day 3의 docker run -p가 필요하다.

실행 명령

repository root에서 실행한다.

```
docker run -d \
  --name paperclip-day3-web \
  -p 18083:80 \
  -v "$PWD/week2/day3/labs/runtime-site/html:/usr/share/nginx/html:ro" \
  nginx:1.27-alpine

docker ps --filter name=paperclip-day3-web
curl -I http://localhost:18083
curl -s http://localhost:18083 | grep runtime-site-v1
docker logs paperclip-day3-web
```

Linux 사전 테스트 결과

docker ps 핵심 출력:

```
0.0.0.0:18083->80/tcp, [::]:18083->80/tcp
```

HTTP header:

```
HTTP/1.1 200 OK
Server: nginx/1.27.5
Content-Length: 1369
```

body 확인:

```
runtime-site-v1
```

포트 읽는 법

표기	의미
18083:80	host 18083 -> container 80
localhost:18083	host에서 접근할 주소
0.0.0.0:18083->80/tcp	모든 host interface의 18083이 container 80으로 publish됨
[::]:18083->80/tcp	IPv6 interface도 publish됨
80/tcp만 보임	host publish 없이 container 내부 port만 노출됨

핵심 유의사항

localhost는 항상 "내가 명령을 실행하는 위치" 기준이다. host terminal에서 localhost:18083은 host 자신을 의미하고, container 안에서 localhost:80은 container 자신을 의미한다.

port 충돌은 host port에서 생긴다. 이미 다른 container나 process가 host 18083을 쓰고 있으면 같은 host port로 다시 publish할 수 없다. 이때 container port 80을 바꾸는 것이 아니라 host port를 18084처럼 바꾸는 방식으로 해결할 수 있다.

curl -I는 header만 확인한다. HTTP 200이 나와도 body가 기대한 페이지인지 모를 수 있다. 그래서 curl -s ... | grep runtime-site-v1 같은 body check를 함께 수행한다.

자주 놓치는 지점

놓치는 지점	증상	확인
host/container port 순서 혼동	localhost:80으로 접속	docker ps PORTS
EXPOSE만 있으면 열린다고 생각	browser 접속 실패	docker run -p 확인
container ID 출력만 보고 성공 처리	container가 바로 죽을 수 있음	docker ps, docker logs
header만 확인	예전 페이지를 볼 수 있음	body text grep
cleanup 생략	다음 실습 port 충돌	docker rm -f

wrong port drill

정상 실행 후 일부러 잘못된 port로 접근한다.

```
curl -I http://localhost:80
curl -I http://localhost:18083
```

비교할 것:

- host 80에 아무 service가 없으면 실패할 수 있다.
- host 18083은 container 80으로 publish되어 HTTP 200이 나온다.
- 실패는 nginx 내부 port가 달렸다는 뜻이 아니라 host 접근 port가 틀렸다는 뜻일 수 있다.

운영 관점

port binding은 README handoff에서 반드시 명시해야 한다. "nginx container 실행"이라고만 쓰면 다른 사람이 어느 port로 접속해야 하는지 알 수 없다.

좋은 handoff:

```
container: paperclip-day3-web
image: nginx:1.27-alpine
host port: 18083
container port: 80
check: curl -I http://localhost:18083
```

나쁜 handoff:

```
nginx 실행됨
```

기록 템플릿

```
## Lesson 1 Port Evidence
- container name:
- image:
- host port:
- container port:
- docker ps PORTS:
- HTTP status:
- body check:
- wrong port 결과:
- cleanup:
```

마무리 점검

아래 문장을 완성한다.

```
-p 18083:80에서 18083은 ____ port이고, 80은 ____ port다.
browser는 ____로 접속한다.
```

container 내부 nginx는 ____번 port를 듣는다.

cleanup

```
docker rm -f paperclip-day3-web
```

평가 기준

기준	2점 evidence
port 구분	host/container port를 정확히 설명했다
실행	container를 -p로 실행했다
HTTP	curl -I와 body grep을 확인했다
장애	wrong port 실패를 설명했다
정리	container를 제거했다

공식 근거 링크

- Docker run reference: <https://docs.docker.com/reference/cli/docker/container/run/>
- Docker networking: <https://docs.docker.com/engine/network/>